Linguaggi di Programmazione

Semantica statica, semantica dinamica, struttura di un compilatore

Roberto Gorrieri

Alma Mater Studiorum - Università di Bologna

Semantica statica e semantica dinamica

Vincoli sintattici contestuali

Con grammatiche libere (o BNF) non si descrivono tutti gli aspetti sintattici rilevanti, perché queste non sono in grado di descrivere vincoli che dipendono dal contesto (ad esempio, dalle dichiarazioni).

Esempi di vincoli sintattici contestuali

- **Dichiarazioni di variabili**: una variabile in uso deve prima essere stata dichiarata.
- **Type checking**: ad esempio, compatibilità di tipo in un assegnamento; infatti, in `x := e`, la variabile `x` e l'espressione `e` devono avere lo stesso tipo.
- **Corrispondenza tra dichiarazione e chiamata di procedure o funzioni**: il numero (e il tipo) dei parametri attuali di una chiamata di procedura deve essere uguale al numero (e al tipo) dei parametri formali della dichiarazione.
- **Visibilità e scope**: verificare l'utilizzo di variabili all'interno del loro campo d'azione.

Vincoli sintattici contestuali (2)

Altri esempi di vincoli sintattici contestuali:

- **Unicità dei nomi**: Impediscono la dichiarazione di due entità con lo stesso nome nello stesso spazio dei nomi (namespace) o blocco, per evitare ambiguità.
- **Correttezza del flusso**: Verificano, ad esempio, che un'istruzione `return` sia presente in tutte le ramificazioni di una funzione che deve restituire un valore, o che `break` sia usato solo dentro un ciclo.

Possibili soluzioni al problema? Ci sono due alternative:

- Usare strumenti più potenti, come le grammatiche dipendenti dal contesto.
  - Poco pratico: la complessità del problema $w \in L(G)$ dove G è una grammatica dipendente dal contesto è esponenziale nella lunghezza di w e nella taglia di G.
- Usare controlli "ad hoc".
  - Nella costruzione di un compilatore, la fase di analisi semantica fa questi controlli.

Sintassi o Semantica?

I vincoli sintattici contestuali "appartengono" pienamente alla sintassi.
Ma tradizionalmente, nel gergo dei linguaggi di programmazione, si intende per:

- **sintassi**: solo esclusivamente quello che si descrive in BNF (o con grammatiche libere da contesto);
- **semantica**: tutto il resto ... (quindi anche i vincoli sintattici contestuali)

I vincoli sintattici contestuali sono dunque considerati vincoli "semantici", detti di semantica statica, cioè vincoli che possono essere verificati ispezionando il programma senza mandarlo in esecuzione.
Il compilatore delega questi controlli di semantica statica (o sintassi contestuale) alla terza fase, denominata **Analisi Semantica**.
Ma la semantica statica non considera in nessun modo il "significato" (ovvero la semantica dinamica) del programma!

Semantica Statica vs Semantica Dinamica

Per semantica **statica** (o sintassi contestuale) si intende l'insieme di quei controlli che possono essere fatti sul programma senza eseguirlo. Ad esempio, il type checking:

```
int A;
bool B;
A := B; # Errore di tipo!
```

Per semantica **dinamica** si intende una rappresentazione formale dell'esecuzione del programma, la quale può mostrare errori "dinamici", cioè durante l'esecuzione.

```
int A, B;
read(A);
B := 10/A; # Se A = 0?
```

Se si fornisce l'input 0 per A, allora la divisione 10/A dà errore in esecuzione. Non possiamo staticamente sapere se l'errore si verificherà, perché ciò dipende dall'input che l'utente fornirà quando il programma verrà eseguito.

Semantica Dinamica

- Fornire un modello matematico che descriva, indipendentemente dall'architettura su cui il programma verrà eseguito, il **"comportamento"** del programma (ovvero il suo significato).
- L'implementazione di un linguaggio (in cui è scritto il programma) deve rispettare questa "specifica" del linguaggio.

Esempio:

- Consideriamo un semplice comando `x:= x+1` e supponiamo di volerlo eseguire in uno store σ (uno store è una funzione che specifica il valore corrente delle variabili).
- La sua semantica dinamica, in un modello a "grafo" (modello astratto e quindi indipendente dall'architettura), potrebbe essere
  - $\langle x := x + 1, \sigma \rangle \rightarrow \sigma[\sigma(x) + 1/x]$
    ovvero una transizione che ha come effetto l'aggiornamento dello store (σ viene aggiornato associando ad x il valore $\sigma(x) + 1$).

A chi serve la Semantica Dinamica

- Al **programmatore** (per fare analisi del programma):
  - deve sapere esattamente cosa debba fare il suo programma;
  - deve poter dimostrare proprietà del suo programma (ad esempio: "termina sempre per ogni possibile input?")
- Al **progettista del linguaggio**:
  - la semantica dinamica è lo strumento di specifica del linguaggio;
  - serve per poter dimostrare proprietà del suo linguaggio (ad esempio: "è Turing-completo?")
- All'**implementatore del linguaggio**:
  - serve come riferimento per dimostrare la correttezza dell'implementazione.

N.B. Ricorda che un compilatore è corretto quando preserva la semantica (dinamica!) del linguaggio che viene tradotto.
Quindi per dimostrare che un compilatore è corretto, serve avere la semantica dinamica per il linguaggio sorgente e per il linguaggio oggetto.

Come si fa a definire la semantica dinamica?

Due tecniche principali:

- **Operazionale**: tipicamente per mezzo di una macchina astratta a stati e transizioni
  - si costruisce una specie di automa che, passo a passo, mostra l'effetto della esecuzione delle varie istruzioni;
  - enfasi su come si calcola.
- **Denotazionale**: si associa ad ogni programma sequenziale una funzione da input ad output (incluse strutture ausiliarie dette ambiente e memoria);
  - enfasi su cosa si calcola (vengono nascosti i passi intermedi del calcolo).

Vedremo l'uso della semantica operazionale per un semplice linguaggio imperativo, utilizzando la tecnica denominata *Semantica Operazionale Strutturata (SOS)*.

Altri aspetti della descrizione di un linguaggio

- **Pragmatica**: insieme di "regole" (o consigli) sul modo in cui è meglio usare le istruzioni a disposizione ("buone maniere" per linguaggi naturali, "stile di programmazione" per linguaggi artificiali).
- Esempi:
  - Evitare le istruzioni di salto (`GOTO`) quando possibile - altrimenti i programmi diventano presto incomprensibili.
  - Usare le variabili di controllo del `for` solo a quello scopo (per evitare side-effect pericolosi).
  - Scelta della modalità più appropriata di passaggio di parametri ad una funzione (ad esempio: mai usare il passaggio per valore per strutture dati grandissime - spreco di memoria).
  - Scelta tra iterazione determinata (`for` da preferirsi, quando possibile) ed indeterminata (`while`, `repeat`).

Infine... Implementazione

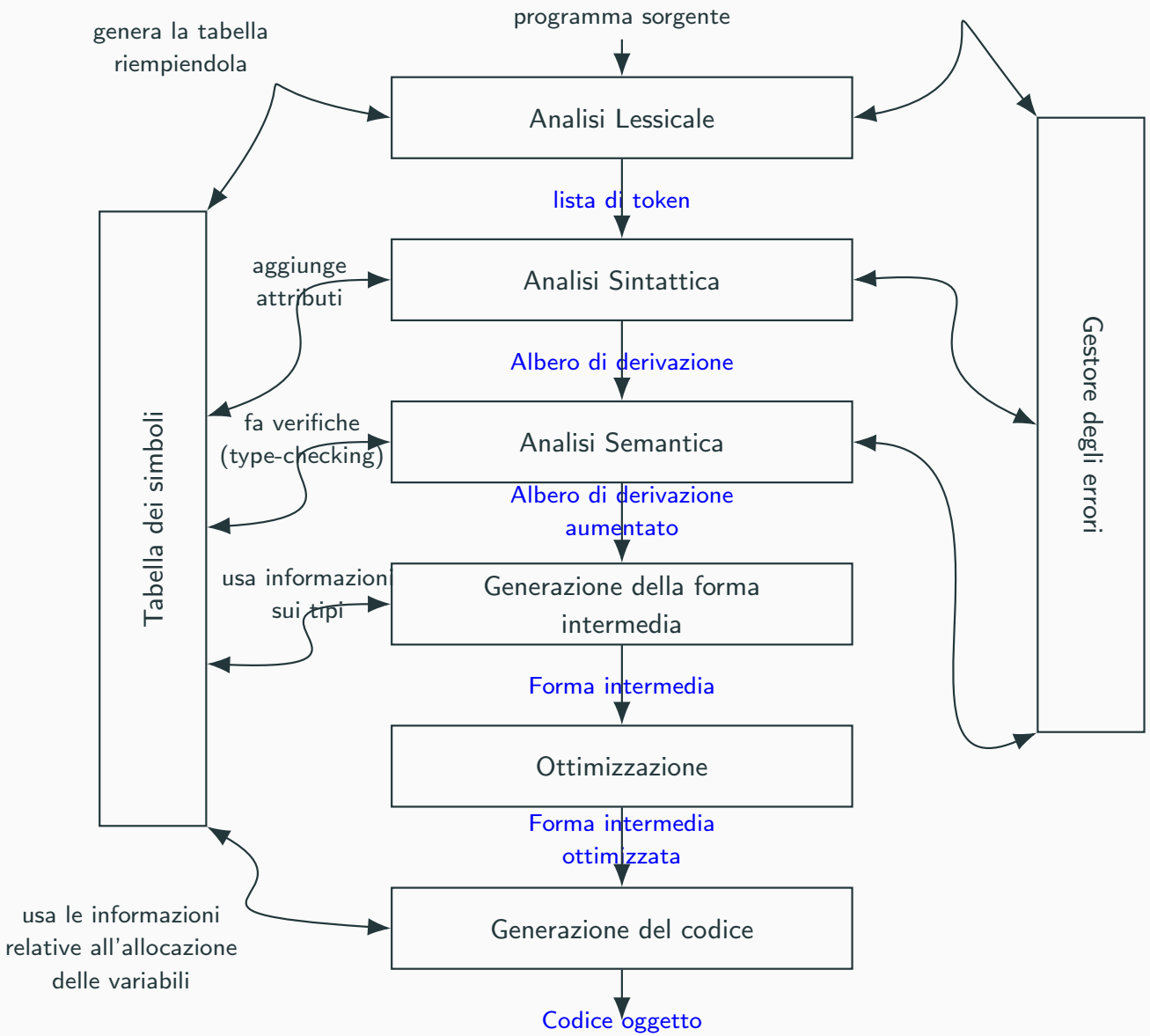
Ovvero scrivere un compilatore (o un interprete) per una macchina ospite già realizzata, costruendo così una macchina astratta per il linguaggio.

- **Correttezza dell'implementazione?**
  Bisogna dimostrare che preserva la semantica dinamica, ovvero che il programma sorgente e quello in oggetto calcolano la stessa funzione (per i linguaggi sequenziali).
- **Costo dell'implementazione?**
  - è il compilatore in grado di produrre codice efficiente?
  - certi costrutti linguistici, scelti dal progettista del linguaggio, sono troppo onerosi in pratica?

E così abbiamo terminato la descrizione dei vari aspetti di un linguaggio, ovvero: sintassi, semantica, pragmatica e implementazione.

Struttura di un compilatore

Struttura di un compilatore



Struttura di un compilatore (2)

- Struttura modulare in sei fasi.
- Le prime tre fasi verificano la correttezza sintattica del programma:
  - ogni errore rilevato non blocca il compilatore, ma produce un opportuno messaggio d'errore;
  - di solito, solo se nessun errore viene rilevato nelle prime tre fasi, allora il compilatore procede alle successive tre fasi (ma non sempre è così).
- Le successive tre fasi generano il codice compilato:
  - prima in una forma intermedia, generata a partire dall'albero di derivazione (o sintattico), poi una forma ottimizzata, infine il codice vero e proprio.
- Le prime quattro fasi sono specifiche del linguaggio sorgente (e, a volte, realizzate in un'unica passata), mentre le ultime due sono orientate al linguaggio oggetto:
  - quindi i moduli delle prime quattro fasi possono essere riusati per produrre un nuovo compilatore verso un altro linguaggio oggetto.

Andiamo ora a descrivere un po' in dettaglio ognuna delle sei fasi in cui si articola un compilatore.

Fasi principali della compilazione: Analisi lessicale

L'**Analizzatore Lessicale (o Scanner)** spezza il programma sorgente nei componenti sintattici primitivi, chiamati tokens (identificatori, numeri, operatori, parentesi, ...)

- Controlla solo che il "lessico" sia ammissibile. Tipici errori rilevati:
  - operatori inesistenti, identificatori non ammessi, simboli non previsti.
- Crea e riempie parzialmente la tabella dei simboli (inserisce gli identificatori di variabili, procedure, funzioni, ...)

Per realizzare uno scanner, avremo bisogno di studiare:

- espressioni regolari (un formalismo usato per descrivere i linguaggi regolari)
- automi finiti (NFA - DFA): uno strumento che permette di riconoscere i linguaggi regolari
- grammatiche regolari (un sotto-tipo di grammatiche libere le cui produzioni sono del tipo $A \to aB$ oppure $A \to a$ oppure $A \to \epsilon$, che generano i linguaggi regolari)

Fasi principali della compilazione: Analisi sintattica

A partire dalla lista di token, generata dallo scanner, l'**Analizzatore Sintattico (o Parser)** produce l'albero di derivazione del programma. Inoltre aggiunge attributi (ad esempio, il tipo) agli identificatori nella tabella dei simboli.
Verifica se le frasi sono sintatticamente corrette; ad esempio, controlla:

- che i comandi siano composti secondo le regole grammaticali:

```
if x=5 then then x:=3      #errore!
```

- che le parentesi usate in espressioni aritmetiche siano bilanciate:

```
(a + (*b)))    #due errori! Quali?
```

Per realizzare un parser, avremo bisogno di studiare:

- grammatiche libere da contesto (già viste, ma le approfondiremo)
- automi a pila (pushdown automata - PDA), soprattutto nella versione deterministica (DPDA)

Fasi principali della compilazione: Analisi semantica

L'**Analizzatore Semantico** esegue dei controlli di semantica statica per rilevare eventuali errori sintattici contestuali (che però vengono chiamati impropriamente errori "semantici"). In particolare:

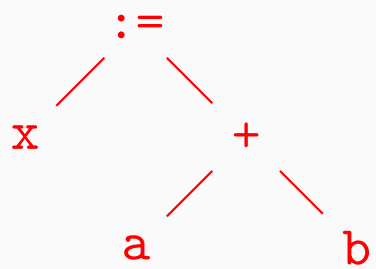
- arricchisce l'albero di derivazione (o sintattico) generato dal parser con informazioni sui tipi;
- fa i controlli sintattici contestuali (verifica i tipi negli assegnamenti, parametri attuali vs formali, dichiarazione ed uso di variabili,...);
- genera eventualmente opportuni messaggi di errore.

Se al termine di queste tre fasi (eseguite in sequenza) non sono stati rilevati errori, allora il compilatore procede con le fasi successive di generazione del codice, altrimenti si blocca restituendo la lista degli errori rilevati.

Fasi principali della compilazione: generazione della forma intermedia

Il **Generatore della Forma Intermedia** genera codice scritto in un linguaggio intermedio, indipendente dall'architettura, facilmente traducibile nel linguaggio macchina di varie macchine diverse.
Caratteristiche:

- **easy to produce - easy to translate**
- utilizza operazioni molto semplici, tipicamente *three-address code* del tipo `x := y + z`
- nel generare codice intermedio, si segue la struttura dell'albero sintattico, ricavato dall'albero di derivazione. Ad esempio, l'istruzione `x := a + b` ha associato questo albero sintattico e produce il codice intermedio qui sotto:



$$\begin{aligned} temp &:= a + b \\ x &:= temp \end{aligned}$$

Fasi principali della compilazione: Ottimizzazione

Si effettua una fase di **ottimizzazione** sul codice intermedio per renderlo più efficiente:

- riduzione di assegnamenti non necessari (come l'ultimo assegnamento nell'esempio precedente);
- riduzione di operazioni non necessarie (vedi esempio finale, in cui la funzione `into-real` viene rimossa);
- rimozione del codice inutile (dead code);
- espansione in linea di chiamate di funzioni;
- fattorizzazione di sotto-espressioni;
- mettere fuori dai cicli sotto-espressioni che non variano;
- ....

Alla fine si ottiene un codice intermedio ottimizzato e pronto per essere tradotto in codice oggetto.

Fasi principali della compilazione: Generazione di codice

In questa fase, il codice intermedio ottimizzato viene tradotto in codice per una specifica architettura, tipicamente in codice assembly o codice macchina rilocabile.

- Viene fatta l'assegnazione delle locazioni di memoria per ogni variabile di programma.
- Ogni istruzione viene tradotta in una sequenza di istruzioni del codice macchina che esegue lo stesso lavoro (cioè tale sequenza di istruzioni preserva la semantica dinamica dell'istruzione originale).
- Vengono fatte ulteriori ottimizzazioni specifiche per quell'architettura:
  - ad esempio: quali variabili conviene tenere nei registri del processore?

Struttura dati condivisa dalle sei fasi: Tabella dei simboli

Tabella dei Simboli
Memorizza le informazioni sui nomi presenti nel programma (identificatori di variabili, funzioni, procedure, ...)

- Viene creata dall'analizzatore lessicale, che la riempie parzialmente.
- Viene arricchita dall'analizzatore sintattico con i tipi delle variabili. Ad esempio:
  - per le matrici, mette come attributo la dimensione ed il tipo degli elementi;
  - per le funzioni, mette come attributo il numero e il tipo dei parametri formali.
- Viene usata dall'analizzatore semantico per fare type-checking.
- Viene usata anche nelle fasi successive per creare il codice.

Esempio di compilazione

Prendiamo un frammento di un programma Pascal

```
var position, initial, rate: real;
position := initial + rate * 60
```

e vediamo come viene trasformata l'istruzione

```
position := initial + rate * 60
```

nelle varie fasi del compilatore.

Esempio continua: analisi lessicale

`position := initial + rate * 60`



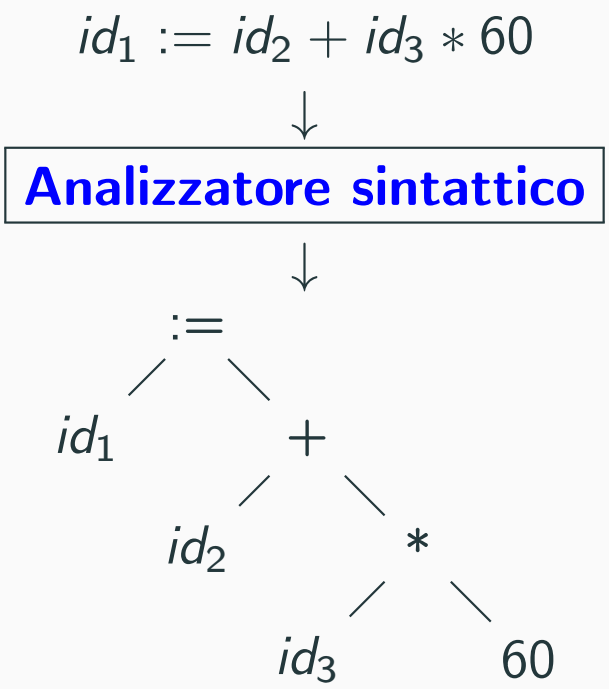
$< id, 1 >< :=>< id, 2 >< op, + >< id, 3 >< op, * >< const, 60 >$

(più brevemente: $id_1 := id_2 + id_3 * 60$)

- Viene prodotta una lista di token:
  - ogni token è racchiuso fra parentesi angolate;
  - il token $< id, 1 >$ indica che si tratta di un identificatore, il cui nome si trova nella posizione 1 della tabella dei simboli (cioè 1,2,3 sono indirizzi nella tabella dei simboli, che contiene i nomi delle tre variabili);
  - $<:=>$ è un token che contiene un simbolo predefinito (assegnamento);
  - $< op, + >$ è un token che specifica che $+$ è un operatore aritmetico;
  - $< const, 60 >$ è un token che specifica che 60 è una costante numerica.

Esempio continua: analisi sintattica

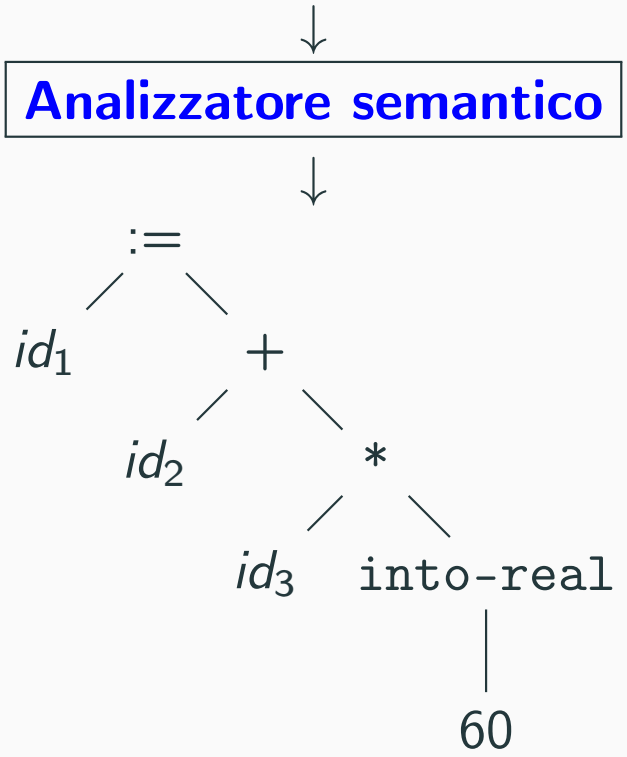
Prende in input la lista di token, qui rappresentata come stringa:



E produce in output questo albero sintattico!

Esempio continua: analisi semantica

Prende in input l'albero sintattico generato dal parser

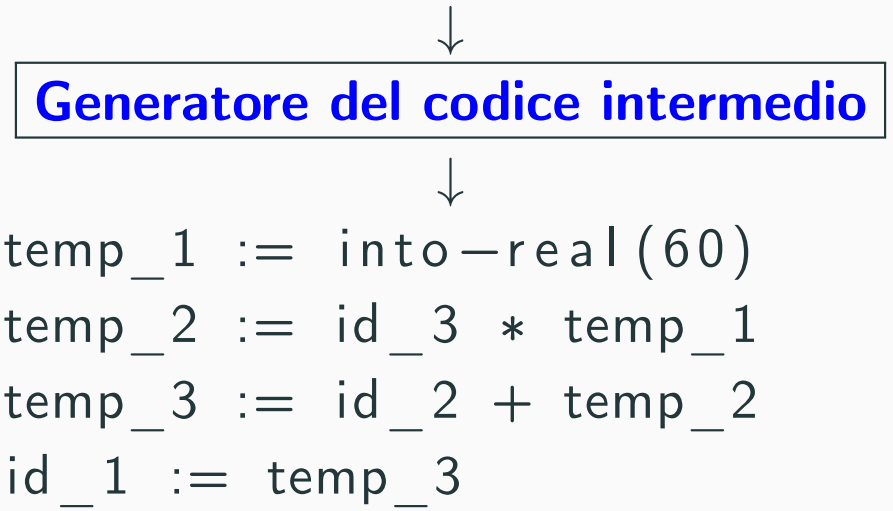


e lo arricchisce in base alle sue informazioni sui tipi:

- poichè tutte le variabili sono di tipo "real", allora tutte le operazioni saranno su "real", ed anche 60 deve essere convertito in "real".

Esempio continua: generazione del codice intermedio

L'albero sintattico arricchito prodotto dall'analizzatore semantico viene usato dal

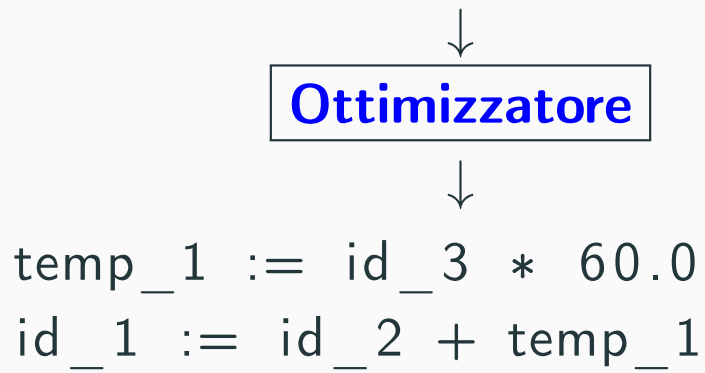


per produrre codice intermedio.

- Usiamo "three-address code" come linguaggio intermedio.
- La generazione del codice segue l'albero sintattico bottom-up.
- Un temporaneo viene generato per memorizzare il risultato di ogni operazione.
- Ci sono istruzioni senza operazioni "vere" (l'ultima).

Esempio continua: ottimizzazione

Il codice intermedio viene dato in pasto all'

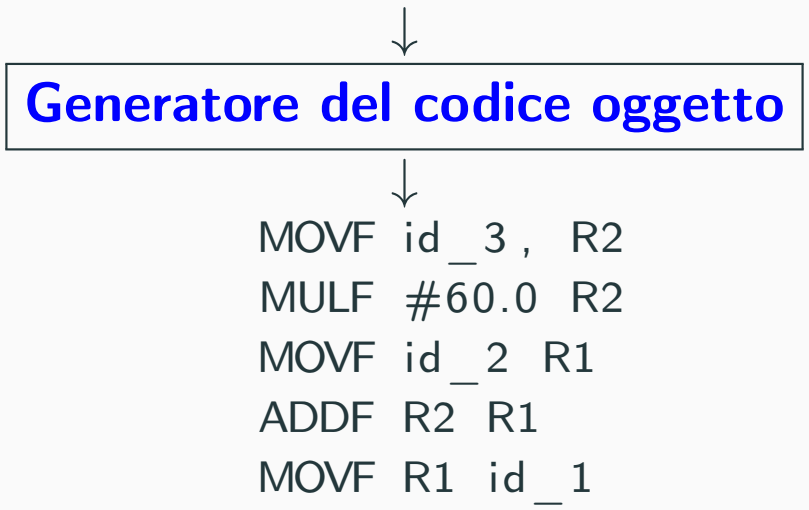


per produrre codice intermedio ottimizzato. Principali ottimizzazioni:

- La conversione da 60 a 60.0 è fatta dal compilatore (non è neccessaria la funzione `into-real`).
- Sono state ridotte le operazioni di assegnamento.

Esempio continua: generazione del codice oggetto

Il codice intermedio ottimizzato viene dato in pasto al



per produrre codice assembly (ma anche codice macchina rilocabile).

- Viene fatta l'allocazione delle variabili ai registri;
- la `F` sta ad indicare che le operazioni *MOV, MUL, ADD* vanno fatte in "floating-point arithmetics";
- l'annotazione ♯ davanti a `60.0` sta ad indicare che quel valore deve essere trattato come costante e non come indirizzo di memoria.